Electro_Neurons_edgeminds2026

Autonomous local research agent built for **EDGE MINDS HACKATHON 2026** by Team **Electro_Neurons**. The project uses **LLaMA 3.2 (1B)** through Ollama to break a research topic into sub-questions, retrieve relevant evidence from local documents, generate concise evidence-grounded answers, and compile a final report without relying on external APIs.

Repository: github.com/adtiwari936/Electro_Neurons_edgeminds2026internship

Overview

This project is designed as a lightweight offline research assistant for edge environments such as the NVIDIA Jetson Orin Nano. Its workflow follows a multi-stage pipeline: topic input, sub-question planning, local document retrieval, reasoning and summarization, and final report generation.

The current implementation uses a simple local retrieval-augmented generation pattern: documents are loaded from a local folder, split into chunks, ranked with keyword scoring, and the top snippets are passed to the language model as context for answer generation.

Features

- Offline-first local research workflow over `.txt` and `.md` files.
- Topic decomposition into 3 to 5 sub-questions using a local LLM, with fallback rule-based planning if needed.
- Lightweight local retrieval over chunked documents using keyword scoring.
- Evidence-grounded answer generation using retrieved snippets only.
- Automatic report export to Markdown and evidence export to JSON for traceability.
- Configurable model, document directory, output directory, chunk size, overlap, and retrieval depth through environment variables.

Architecture

The codebase is organized into small single-purpose modules:

File	Purpose
`main.py`	CLI entry point that accepts a research topic and prints the generated report.
`agent.py`	Orchestrates the full end-to-end pipeline.
`planner.py`	Generates sub-questions from the input topic using the local model, with a fallback path.
`retriever.py`	Loads local `.txt` and `.md` documents, chunks them, scores them, and returns top evidence snippets.
`synthesizer.py`	Generates answers using only the retrieved evidence snippets.
`reporter.py`	Compiles the final report and saves `final_report.md` and `evidence.json`.

File	Purpose
`ollama_client.py`	Creates the Ollama-backed LLM client through LlamaIndex.
`config.py`	Centralizes runtime configuration values from environment variables.
`utils.py`	Helper functions for file reading, chunking, scoring, and directory creation.

How it works

1. The user runs the program with a topic from the command line.
2. `agent.py` loads the local document set from the configured `documents/` folder.
3. `planner.py` creates 3 to 5 sub-questions for the topic.
4. For each sub-question, `retriever.py` searches local chunks and returns the top evidence snippets.
5. `synthesizer.py` answers each sub-question using only those snippets as context.
6. `reporter.py` merges the answers into a final Markdown report and saves supporting evidence to JSON.

Installation

1. Clone the repository

```
git clone
https://github.com/adtiwari936/Electro_Neurons_edgeminds2026internship.git
cd Electro_Neurons_edgeminds2026internship
```

2. Create and activate a virtual environment

```
python -m venv .venv
```

Windows (PowerShell)

```
.\.venv\Scripts\Activate.ps1
```

Linux / macOS

```
source .venv/bin/activate
```

3. Install Python dependencies

```
pip install -r requirements.txt
```

The current LLM integration expects Ollama access through LlamaIndex's Ollama connector.

4. Install and start Ollama

```
ollama pull llama3.2:1b
ollama serve
```

The default configuration expects the model name `llama3.2:1b` and an Ollama server at `http://localhost:11434`.

Project structure

```
Electro_Neurons_edgeminds2026internship/
├── agent.py
├── config.py
├── main.py
├── ollama_client.py
├── planner.py
├── reporter.py
├── requirements.txt
├── retriever.py
├── synthesizer.py
├── utils.py
├── documents/
│   ├── sample1.md
│   └── sample2.txt
├── output/
│   └── .gitkeep
└── README.md
```

The program scans the `documents/` folder recursively and currently reads `.txt` and `.md` files only.

Usage

Add local knowledge files to the `documents/` folder, then run:

```
python main.py "impact of climate change on crop yield in India"
```

The CLI accepts one positional argument called `topic`, which is used as the research prompt for the agent.

You can also try related variations of the same theme, for example:

```
python main.py "effects of climate change on Indian agriculture"
python main.py "impact of drought and irregular rainfall on crop yield in India"
python main.py "climate risks for wheat and rice production in India"
```

Output

Each run generates the following files:

- `output/final_report.md` — the compiled research report.
- `output/evidence.json` — serialized evidence snippets used for each sub-question.

Because these filenames are fixed, each new run overwrites the previous output unless the files are copied or renamed manually after execution.

Configuration

The application reads configuration from environment variables with sensible defaults:

Variable	Default	Purpose
`MODEL_NAME`	`llama3.2:1b`	Ollama model to use.
`OLLAMA_URL`	`http://localhost:11434`	Ollama server endpoint.
`DOCUMENTS_DIR`	`documents`	Folder containing local source documents.
`OUTPUT_DIR`	`output`	Folder for generated outputs.
`TOP_K`	`4`	Number of evidence snippets returned per sub-question.
`MAX_SUBQUESTIONS`	`5`	Maximum number of planned sub-questions.
`CHUNK_SIZE`	`700`	Character length for document chunking.
`CHUNK_OVERLAP`	`120`	Overlap between adjacent chunks.

Example:

Windows (PowerShell)

```
$env:MODEL_NAME="llama3.2:1b"
$env:TOP_K="3"
python main.py "effects of climate change on Indian agriculture"
```

Linux / macOS

```
export MODEL_NAME="llama3.2:1b"
export TOP_K="3"
```

```
python main.py "effects of climate change on Indian agriculture"
```

RAG note

This project already uses a lightweight local retrieval-augmented generation workflow. It retrieves relevant chunks from local files and passes those snippets into the LLM prompt for answer generation, but it does **not** yet use embeddings, vector databases, or semantic similarity search.

That makes the current system a simple keyword-based local RAG pipeline rather than a full embedding-based RAG stack.

Jetson Orin Nano fit

The project was designed with edge deployment in mind, and the problem statement explicitly targets the NVIDIA Jetson Orin Nano for efficient offline inference with a 1B-parameter model.

The lightweight architecture helps this fit edge hardware better than larger LLM pipelines because retrieval is local, outputs are short, and model size remains small.

Current limitations

- Retrieval is based on keyword overlap, so paraphrased queries may perform unevenly.
- Only `.txt` and `.md` local documents are supported in the current implementation.
- Output files are overwritten on every run unless preserved manually.
- Planner and summarizer quality depend heavily on the small local model and prompt design.
- The iterative agent loop described in the project vision is not yet fully implemented in code; the current pipeline plans once, answers once per sub-question, and compiles once.

Future improvements

Potential next steps already identified in the project statement include:

- Document chunking improvements.
- Embedding-based retrieval.
- Vector search integration.
- Confidence scoring for evidence.
- Persistent memory and stronger agent state tracking.
- Prompt optimization or fine-tuning for LLaMA 3.2 (1B).

Demo flow

A typical demo run looks like this:

1. Place local source files in `documents/`.
2. Start Ollama and ensure `llama3.2:1b` is available.
3. Run the agent with a topic.
4. Review the generated Markdown report in `output/final_report.md`.
5. Inspect `output/evidence.json` to verify which snippets were retrieved.

Team and event

- **Team:** Electro_Neurons
- **Event:** EDGE MINDS HACKATHON 2026
- **Project title:** Autonomous Local Research Agent Using LLaMA 3.2 (1B)